

תרגיל בית 1:

שאלה 1:

- א. עבור קלט של המספר 5432345 יודפס הערך '26' (סכום הספרות של הקלט).
- ב. בהינתן מספר שלם וחיובי כלשהו n , התוכנית תדפיס את ערך סכום הספרות של המספר n .
- ג. עבור כל קלט של מספר שלילי n , התוכנית תדפיס '0'. זה משום שהתנאי כניסה ללולאת `while` אשר הגדרנו הינה כי $n > 0$ אך במקרה של הסעיף `n` מספר שלילי. לפיכך לא יערך שום שינוי בערך המשתנה `res` שהוגדרה בהתחלה והוא יודפס בסוף התוכנית עם ערכו ההתחלתי - '0'.
- ד. כאשר אנו רוצים להכניס קלט כביטוי אריתמטי כמו $4^{**}111 + 25$ מתעוררת שגיאה שמעלה הפקודה הכוללת את פעולת ההמרה (`int()`) - פקודת ההמרה מטיפוס `string` לטיפוס `int` היא המעלה את השגיאה. היא מעלה שגיאה זו משום שהפונקציה המובנית `int()` לא יכולה לקבל (וקורסת אם היא מקבלת) מחרוזות הכוללות תווים השונים מספרות ולפיכך לא יכולה לבצע פעולות אריתמטיות שבתוך המחרוזת. בהעברתנו לפעולה `int()` מחרוזת עם תווים שהם לא ספרות ('+', ' ', '*', '*') גרמו לפעולה להעלות את השגיאה.
- ה. הפונקציה `eval()` מקבלת מחרוזת ומעבדת את תוכן המחרוזת כאילו פקודה שנכתבה בפייתון. באמצעותה, ניתן לבצע פעולות אריתמטיות רגילות של פייתון שנמצאים בתוך מחרוזת (או כל פעולה בפייתון לצורך העניין). הפונקציה מעבדת את תוכן המחרוזת כאילו שנכתבה כשורת קוד בפייתון ולכן נשתמש בה לביצוע הפעולה האריתמטית מהסעיף הקודם. כאשר נשתמש בפעולה זו, הפלט עבור הקלט $4^{**}111 + 25$ יהיה 341.
- ו. כאשר נשנה את תנאי הלולאה ל- $num \geq 0$ נקבל כי התוכנית לא מדפיסה כלום וזה משום שהלולאה הופכת ללולאה אינסופית וזה משום שהתנאי עצירה שלנו ($num < 0$) לא מתקיים מעולם. מפני שלאחר חלוקת המספר בעשר ועיגול מטה (`//`) לצורך "מחיקת" ספרת האחדות בהכרח `num` לבסוף יהיה שווה לאפס ותנאי ריצת הלולאה עדיין מתקיים. בהמשך הריצה מבוצעת אותה חלוקה למספר אפס (הנשאר אפס) ולפיכך אף פעם לא נגיע לתנאי העצירה ונישאר בלולאה אינסופית.

שאלה 2:

פונקציות הקיימות במחלקת `str` שלא נמצאות במחלקת `list` - `capitalize` (כל מילה במחרוזת שמתחילה עם אות קטנה, האות הראשונה הופכת לאות גדולה), `upper` (כל האותיות הקטנות במחרוזת הופכות לאותיות גדולות), `swapcase` (כל אות קטנה במחרוזת הופכת לאות גדולה ואילו כל אות גדולה הופכת לאות קטנה):

1.

```
>>> string = "abcd"
>>> str.capitalize(string)
'Abcd'
>>> string.capitalize()
'Abcd'
```

2.

```
>>> string = "abcd"
>>> str.upper(string)
'ABCD'
>>> string.upper()
'ABCD'
```

3.

```
>>> string = "abcd"
>>> str.swapcase(string)
'ABCD'
>>> string.swapcase()
'ABCD'
```

פונקציות הקיימות במחלקת list שלא קיימות במחלקת str - reverse (הופך את סדר האיברים ברשימה), סקס (מוציא את האיבר האחרון מהרשימה ומחזיר אותו), append (מוסיף איבר לרשימה):

1.

```
>>> my_list = ['a', 'b', 'c', 'd']
>>> list.reverse(my_list)
>>> my_list
['d', 'c', 'b', 'a']
>>> my_list.reverse()
>>> my_list
['a', 'b', 'c', 'd']
```

2.

```
>>> my_list = ['a', 'b', 'c', 'd']
>>> list.pop(my_list)
'd'
>>> my_list
['a', 'b', 'c']
>>> my_list.pop()
'c'
>>> my_list
['a', 'b']
```

3.

```
>>> my_list = ['a', 'b', 'c', 'd']
>>> list.append(my_list, 'e')
>>> my_list
['a', 'b', 'c', 'd', 'e']
>>> my_list.append('f')
>>> my_list
['a', 'b', 'c', 'd', 'e', 'f']
```

שאלה 4:

א' + ב':

1600**2	800**2	400**2	200**2	
0.00050800000000000084	0.0001860000000000001948	6.2999999999997974e-05	2.900000000000001247e-05	פתרון 1
5.0999999999996774e-05	2.900000000000001247e-05	2.29999999999995246e-05	1.400000000000014001e-05	פתרון 2
1.49999999999987246e-05	8.999999999981245e-06	8.00000000000008E-06	6.00000000000060005e-06	פתרון 3

ב. דרך הפיתרון השלישית אכן יותר יעילה מבחינת זמן ריצה בהשוואה לשני הפתרונות האחרים. כפי שניתן לראות בטבלה המוצגת למעלה, אפילו זמן הריצה של הרצת תוכנית הפיתרון השלישי והכנסה כקלט את המספר הגדול ביותר (2**1600) קצר הרבה יותר מזמן הריצה של הפתרון הראשון והשני עם המספר הקטן ביותר (2**200).

ג.

100500070030040	100503070230040	100503070235042	100503070235742	
9.0000000000036756e-06	9.0000000000036756e-06-05	8.00000000000008E-06	8.00000000000008E-06	פתרון 1
8.00000000000008E-06	8.00000000000008E-06	8.00000000000008E-06	6.999999999979245e-06	פתרון 2
6.00000000000060005e-06	6.999999999979245e-06	6.999999999979245e-06	5.999999999950489e-06	פתרון 3

כפי שניתן לראות בטבלת התוצאות המוצגת למעלה, אין קשר בין מספר האפסים בקלט לבין זמן הריצה של כל אחד מהפתרונות. בחרתי ארבעה מספרים בעלי 15 ספרות כל אחד, כאשר לכל אחד מהם יש מספר שונה של אפסים (מימין לשמאל - 5 אפסים, 6 אפסים, 8 אפסים ו-10 אפסים) כאשר הקפדתי שלא יהיה קשר בין מספר האפסים לגודל המספר ולכן כדי שתנאי הניסוי יהיו אחידים, ההבדלים בין ערכי המספרים זניחים ביותר והמספרים דומים מאוד בגודלם. כפי שניתן לראות בתוצאות למרות היחס הישר בעליית מספר האפסים במספרים השונים, כל תוצאות זמן הריצה של כל פיתרון די זהות ולכן אין כל קשר בין מספר האפסים (הפלט והדבר הנבדק בתוכנית) לבין זמן הריצה. כאשר אנו מסתכלים על טבלת התוצאות של סעיף זה וטבלת התוצאות של סעיף א' וב' אנו יכולים להגיע למסקנה כי מה שמשפיע על זמן הריצה הינו גודל המספר ולא מספר ספרות האפסים במספר. כלומר, ככל שהמספר יותר גדול, זמן הריצה יותר ארוך ולהיפך.

ד. להערכת, הרצת הלולאה הפשוטה המצורפת לסעיף זה תיקח זמן רב מאוד, אף הרבה יותר מתוחלת החיים של המחשב והאנשים המבצעים את החישוב וככל הנראה שהלולאה לא תפסיק לרוץ עד שיעבור זמן רב מאוד.

הגתי להערכה זו בשל מספר הנחות. ראשית, כפי שניתן לראות בפיתרון השני בסעיף א' בעת השימוש במספר הקטן ביותר ($2^{**}200$), ביצוע איטרציה על מספר מועט יחסית של איברים ($61 =$ מספר ספרות המספר), לוקח ללולאה זמן מאוד מועט להסתיים. לעומת זאת, במקרה זה, יש לבצע איטרציה על $2^{**}100$ איברים מפני שמבצעים איטרציה מ-0 עד לגודל המספר המדובר, דבר שיקח ככל הנראה גודל זמן בלתי סביר.

כמו כן, ניתן לבצע לולאה זו על מספרים קטנים הרבה יותר ולחשב באמצעות מנת המספר עם הנתון עם המספרים הקטנים את זמן הריצה של התוכנית באופן גס. לצורך כן, כתבתי תוכנית שבודקת את זמן הריצה של הקוד הנתון בסעיף 100 פעמים ומחשבת את ממוצע הזמנים (לצורך דיוק) אך החלפתי את המספר הנתון ($2^{**}100$) במספר קטן הרבה יותר ($2^{**}10$). יצא לי כעת כי זמן הריצה הממוצע הוא 0.00269 שניות. לאחר מכן, מצאתי את היחס בין המספר הנתון ($2^{**}100$) לבין המספר שהצבתי באמצעות מנה פשוטה. יחס זה הכפלתי בזמן הריצה הממוצע (0.00269) ויצא לי כי זמן הריצה בהערכה גסה של הלולאה הנתונה הוא מספר שנים בעל 15 אפסים.

לאחר מכן, רציתי לוודא כי שיטה זו עובדת באופן גס וחישבתי את זמן הריצה של $2^{**}20$ באמצעות השיטה ולאחר מכן באמצעות התוכנית ויצא זמן כמעט זהה בשני החישובים. הלולאה של הפתרון השני רצה בזמן הרבה יותר קצר מפני שהיא מבצעת איטרציה על הספרות (מספר התווים במחרוזת המספר) ולא מ-0 עד לגודל המספר (למשל, למספר $2^{**}200$ היא מבצעת איטרציה על 61 איברים ולא מ-0 עד המספר עצמו).